Name: Vanshika Rao
Section: B4
Batch: B2
Roll No.: 28

PRACTICAL 0

Software Engineering UML Tools

1. Use Case Diagram:

A Use Case Diagram represents the functional requirements of a system from the user's perspective. It shows how different users (actors) interact with the system through various use cases. It helps identify the system's functionalities and external interactions.

Purpose:

- Capture functional requirements
- Identify users and system interactions
- Provide a high-level view of the system

Main Elements:

- Actor
- Use Case
- System Boundary
- Association
- Include
- Extend
- Generalization

2. Class Diagram:

A Class Diagram illustrates the static structure of a system by showing classes, their attributes, methods, and the relationships between them. It serves as the blueprint for object-oriented software design.

Purpose:

- Model the structure of a system
- Define relationships among classes
- Support object-oriented programming

Main Elements:

- Class
- Attributes
- Methods
- Association
- Inheritance
- Aggregation
- Composition

- Dependency

3. Object Diagram:

An Object Diagram is an instance-level representation of a Class Diagram. It shows specific objects and their relationships at a particular moment in time.

Purpose:

- Visualize real instances of classes
- Validate class diagrams
- Understand object relationships

Main Elements:

- Objects
- Attribute Values
- Links

4. Sequence Diagram:

A Sequence Diagram shows how objects interact over time through the exchange of messages. It focuses on the chronological order of interactions.

Purpose:

- Model dynamic behaviour
- Describe method calls
- Show communication between objects

Main Elements:

- Lifeline
- Activation
- Messages
- Actor
- Object
- Return Message

5. Communication Diagram:

A Communication Diagram (formerly Collaboration Diagram) focuses on how objects communicate with each other using messages while emphasizing their structural relationships.

Purpose:

- Show object collaboration
- Represent message flow
- Understand communication paths

Main Elements:

- Objects
- Links
- Messages
- Sequence Numbers

6. Activity Diagram:

An Activity Diagram models the workflow of a system or business process. It represents activities, decisions, loops, and parallel processes.

Purpose:

- Model workflows
- Represent business processes
- Show parallel activities

Main Elements:

- Initial Node
- Activity
- Decision Node
- Merge Node
- Fork
- Join
- Final Node

7. State Machine Diagram:

A State Machine Diagram represents the different states of an object during its lifetime and the events that trigger transitions between those states.

Purpose:

- Model object behaviour
- Represent state changes
- Describe event-driven systems

Main Elements:

- State
- Transition
- Event
- Initial State
- Final State

8. Component Diagram:

A Component Diagram illustrates the physical organization of software components and their dependencies. It helps in understanding the system architecture.

Purpose:

- Model software architecture
- Show reusable components
- Represent dependencies

Main Elements:

- Component
- Interface

- Dependency
- Package

9. Deployment Diagram:

A Deployment Diagram represents the physical deployment of software artifacts on hardware nodes. It shows how software is distributed across servers, devices, and networks.

Purpose:

- Model system deployment
- Show hardware configuration
- Represent network architecture

Main Elements:

- Node
- Device
- Artifact
- Communication Path

10. Package Diagram:

A Package Diagram organizes related UML elements into packages, making large systems easier to understand and maintain.

Purpose:

- Organize system components
- Improve modularity
- Manage dependencies

Main Elements:

- Package
- Dependency
- Nested Package

11. Composite Structure Diagram:

A Composite Structure Diagram shows the internal structure of a class or component, including its parts, ports, and connectors.

Purpose:

- Describe internal architecture
- Show collaborations within a class
- Model complex structures

Main Elements:

- Part
- Port
- Connector
- Collaboration

12. Timing Diagram:

A Timing Diagram focuses on changes in object states over time. It is mainly used in real-time and embedded systems where timing is critical.

Purpose:

- Analyse timing constraints
- Model real-time systems
- Show state changes over time

Main Elements:

- Lifeline
- State
- Time Axis
- Duration Constraint

13. Interaction Overview Diagram:

An Interaction Overview Diagram combines the features of Activity Diagrams and Sequence Diagrams to provide a high-level overview of system interactions.

Purpose:

- Model complex interactions
- Coordinate multiple sequence diagrams
- Represent interaction workflows

Main Elements:

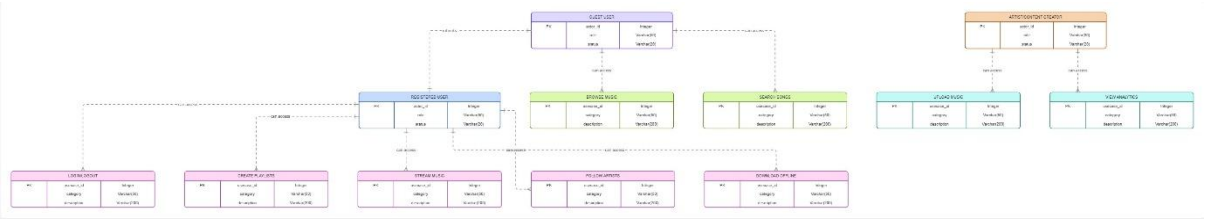
- Initial Node
- Interaction Node
- Decision Node
- Merge Node
- Final Node

Use Case diagram

A Use Case Diagram is a UML diagram that shows how users (actors) interact with a system and the functionalities (use cases) it provides. It gives a high-level view of the system from the user's perspective and helps define system requirements and scope.

- Represents actors (users or external systems) and the use cases they interact with.
- Shows relationships between users and system functionalities, making requirements easier to understand.

SPOTIFY USE CASE DIAGRAM

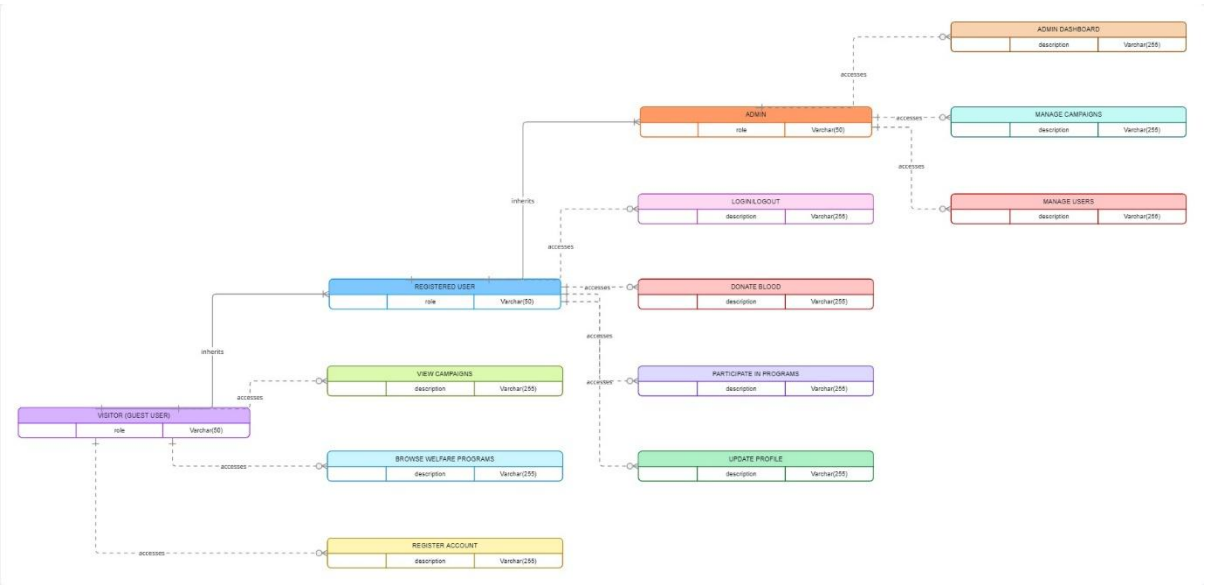


Observations:

- **User Journey:** The flow from Guest to Registered User shows how Spotify encourages visitors to sign up by offering more features after registration.
- **Feature Access:** Guests can browse and search music, while registered users get full access to streaming, playlists, offline listening, and following artists. This is Spotify's freemium model.
- **Two Types of Users:** Spotify serves both listeners and artists. Listeners enjoy music, while artists can upload songs and view analytics.
- **Main Feature:** Stream Music is the central feature, with playlists, follows, and offline listening supporting the overall music experience.
- **User Engagement:** Features like following artists, creating playlists, and streaming music encourage users to return regularly. Artists also use analytics to improve their content.
- **Simple Design:** The diagram uses only 3 actors and 9 use cases, making it easy to understand while covering Spotify's main functionality.

PROJECT USE CASE DIAGRAM

Feel Good Foundation : Web based



Observations:

- **User Journey:** The flow from Visitor to Registered User and then Admin shows increasing levels of access and responsibilities within the system.
- **Feature Access:** Visitors can browse welfare programs, view campaigns, and register. Registered users can log in, donate blood, participate in programs, and update their profiles.
- **Role-Based Access:** The system has three user roles—Visitor, Registered User, and Admin. Each role has different permissions, ensuring users only access relevant features.
- **Admin Functions:** Admins manage users, campaigns, and the dashboard, helping keep the platform organized and running smoothly.
- **Core Purpose:** The main focus of the system is to encourage blood donation and participation in welfare programs, while allowing users to manage their profiles and activities.
- **Simple Design:** With 3 actors and a small set of key use cases, the diagram clearly represents the system's main functionality without unnecessary complexity.

Software Management Tools

1. Jira
Description:
Jira is a project management and issue-tracking tool developed by Atlassian. It helps teams plan, assign, track, and manage software development tasks using Agile methodologies like Scrum and Kanban.
Purpose:
- Track bugs and issues
- Plan Agile projects
- Monitor project progress

Key Features:
- Sprint Planning
- Kanban & Scrum Boards
- Issue Tracking
- Reports & Dashboards

2. Trello
Description:
Trello is a visual project management tool that organizes tasks using boards, lists, and cards. It is easy to use and suitable for individuals and teams.
Purpose:
- Organize tasks
- Track project progress
- Improve team collaboration

Key Features:

- Boards
- Lists
- Cards
- Checklists
- Due Dates

3. Asana

Description:

Asana is a work management platform that helps teams organize, assign, and monitor tasks and projects from start to finish.

Purpose:

- Manage projects
- Assign responsibilities
- Track deadlines

Key Features:

- Task Management
- Project Timeline
- Calendar View
- Team Collaboration

4. Microsoft Project

Description:

Microsoft Project is a project management software used to plan, schedule, and monitor large projects efficiently.

Purpose:

- Create project schedules
- Allocate resources
- Monitor project timelines

Key Features:

- Gantt Charts
- Resource Management
- Scheduling
- Budget Tracking

5. GitHub

Description:

GitHub is a cloud-based platform that hosts Git repositories and allows developers to collaborate on software projects.

Purpose:

- Store source code
- Collaborate with developers
- Manage version control

Key Features:

- Git Repositories
- Pull Requests
- Branching
- Issue Tracking

6. GitLab
Description:
GitLab is a DevOps platform that provides source code management, CI/CD pipelines, and project management tools in one place.
Purpose:
- Manage code repositories
- Automate software delivery
- Collaborate with teams

Key Features:
- Version Control
- CI/CD
- Issue Tracking
- Merge Requests

7. Azure DevOps
Description:
Azure DevOps is Microsoft's platform for software development, testing, deployment, and project management.
Purpose:
- Manage development lifecycle
- Automate builds and releases
- Track project progress

Key Features:
- Azure Boards
- Repositories
- Pipelines
- Test Plans

8. Monday.com
Description:
Monday.com is a work management platform used to plan projects, assign tasks, and monitor team progress through customizable workflows.
Purpose:
- Manage projects
- Improve collaboration
- Track deadlines

Key Features:
- Dashboards

- Automation
- Task Assignment
- Timeline View

9. ClickUp
Description:
ClickUp is an all-in-one productivity and project management tool that combines task management, documentation, goals, and collaboration.
Purpose:
- Organize projects
- Manage tasks
- Improve productivity

Key Features:
- Task Lists
- Docs
- Time Tracking
- Dashboards

10. Basecamp
Description:
Basecamp is a team collaboration and project management tool that helps organize work, discussions, schedules, and files.
Purpose:
- Team communication
- Project organization
- File sharing

Key Features:
- Message Boards
- To-do Lists
- Schedules
- File Storage

Software Testing Tools

1. Selenium
Description:
Selenium is an open-source automation testing tool used to test web applications across different browsers and operating systems.
Purpose:
- Automate web application testing
- Perform cross-browser testing

- Reduce manual testing effort

Key Features:

- Browser Automation
- Multiple Language Support
- Cross-Browser Testing
- Parallel Testing

2. JUnit

Description:

JUnit is a unit testing framework for Java applications. It helps developers write and run repeatable tests to verify code functionality.

Purpose:

- Perform unit testing
- Detect coding errors early
- Improve code quality

Key Features:

- Test Cases
- Assertions
- Test Suites
- Automated Testing

3. TestNG

Description:

TestNG is a Java testing framework inspired by JUnit, offering advanced features like parallel execution and test grouping.

Purpose:

- Execute automated tests
- Manage complex test cases
- Generate detailed reports

Key Features:

- Parallel Testing
- Test Groups
- Data Providers
- HTML Reports

4. Apache JMeter

Description:

Apache JMeter is an open-source tool used for performance and load testing of web applications, APIs, and databases.

Purpose:

- Measure application performance
- Perform load testing
- Identify bottlenecks

Key Features:

- Load Testing
- Stress Testing
- Performance Reports
- API Testing

5. Postman

Description:

Postman is a popular API testing tool used to design, test, and document REST and GraphQL APIs.

Purpose:

- Test APIs
- Automate API testing
- Debug API requests

Key Features:

- API Requests
- Collections
- Automated Tests
- Environment Variables

6. Appium

Description:

Appium is an open-source tool used for automating mobile application testing on Android and iOS devices.

Purpose:

- Test mobile applications
- Automate Android and iOS testing
- Perform cross-platform testing

Key Features:

- Mobile Automation
- Cross-Platform Support
- Multiple Language Support
- Real Device Testing

7. Cypress

Description:

Cypress is a modern testing framework designed for fast and reliable testing of web applications.

Purpose:

- Perform end-to-end testing
- Test web applications
- Improve application reliability

Key Features:

- Real-Time Reload
- Automatic Waiting
- Debugging Tools
- End-to-End Testing

8. LoadRunner
Description:
LoadRunner is a performance testing tool used to evaluate how applications perform under heavy user loads.
Purpose:
- Conduct performance testing
- Measure system scalability
- Identify performance issues

Key Features:
- Load Testing
- Stress Testing
- Performance Monitoring
- Detailed Analysis

9. SoapUI
Description:
SoapUI is a testing tool used for functional and API testing of SOAP and REST web services.
Purpose:
- Test web services
- Validate APIs
- Automate API testing

Key Features:
- SOAP & REST Testing
- Functional Testing
- Load Testing
- Security Testing

10. Bugzilla
Description:
Bugzilla is an open-source bug tracking and defect management tool used to report, monitor, and resolve software issues.
Purpose:
- Track software bugs
- Manage defects
- Improve issue resolution

Key Features:
- Bug Tracking
- Issue Reporting

- Workflow Management
- Email Notifications